YOLO를 활용한 얼굴 자동 모자이크 서비스

프로젝트 기획서 · 사진·영상 속 얼굴을 자동으로 찾아 가려 주는 개인정보 보호 서비스

항목	내용
팀 구성	이유노(A: 모델·데이터·평가), 조영준(B: 모자이크·영상), 나예린(C: 웹·인물 제외)
기간	2주 (10일)
핵심 기술	YOLO11, OpenCV, ffmpeg, InsightFace, 웹 데모(Streamlit·Gradio·Vercel 중 선택)
학습 환경	Google Colab 또는 학원 GPU 서버
산출물	사진·영상·실시간 영상 모자이크 + 웹 데모 + 특정 인물 제외(도전)

1. 프로젝트 개요

사진이나 영상 속 사람 얼굴을 **YOLO로 자동 탐지**하고 해당 영역에 **모자이크**를 적용하는 서비스입니다. SNS·유튜브에 올리는 콘텐츠 속 행인 얼굴을 자동으로 가려 **초상권 침해를 예방**하는 것이 목적입니다. 사진·영상 파일뿐 아니라 **실시간 영상 주소**를 넣어도 동작하며, **특정 인물(본인 등)만 모자이크에서 제외**하는 기능을 더해 일반 모자이크 도구와 차별화합니다.

2. 이 주제를 추천하는 이유

평가 항목	내용	판단
문제 명확성	초상권·개인정보 보호라는 실생활 문제를 직접 해결	높음
구현 난이도	클래스가 face 1개뿐이고, 탐지 결과에 모자이크만 씌우면 되어 후처리가 단순	낮음~중간
일정 적합성	사전학습 얼굴 모델로 첫날부터 세 명이 동시에 작업 가능, 직접 학습 모델은 나중에 가중치만 교체	적합
시연 효과	전/후 비교가 한눈에 보이고, 실시간 영상으로 라이브 시연 가능	높음
차별점	사전학습 vs 직접 학습 성능 비교, 학습 없이 사진·영상만으로 등록하는 인물 제외 기능	있음
주요 리스크	AI Hub 승인 지연, 작은 얼굴 누락, 영상 소리 손실, 실시간 지연, 인물 제외 오인식	대응책 마련 (11장)

3. 타겟층

- 유튜버·영상 크리에이터 (거리 촬영, 브이로그, 라이브 방송)
- SNS에 사진을 올리는 일반 사용자
- CCTV 영상을 외부에 제공해야 하는 기관
- 언론사 (보도 사진·영상의 제3자 얼굴 처리)

4. 기능 범위 (우선순위)

단계	기능	비고
1단계 (필수)	사진(jpg, png) 얼굴 탐지 및 모자이크 / 영상(mp4) 처리 (원본 소리 유지) / 실시간 영상 주소(RTSP·HTTP 스트림, 유튜브 라이브) 처리 / 웹 데모 / 얼굴 수 표시 (사진은 탐지 수, 영상은 추적 기반 등장 인원 수)	프로젝트의 핵심 완성 범위
2단계 (도전)	특정 인물 얼굴 제외: 제외할 인물의 사진 또는 영상을 등록하면 자동으로 얼굴 특징을 저장하고, 탐지된 얼굴과 비교해 일치하면 모자이크 생략. **별도 학습 없음**	2일 차부터 병행 개발, 7일 차에 계속 여부 판단
확장 아이디어	모자이크 방식 선택 (블러·픽셀·이모지), 번호판 모자이크, 얼굴 분할(segmentation) 모델 비교	발표 시 향후 계획으로 제시

특정 인물 제외 동작 방식

얼굴 인식 모델(InsightFace)은 이미 대규모 얼굴 데이터로 학습되어 있어서, 우리 팀이 따로 학습할 필요가 없습니다. 얼굴을 넣으면 그 사람을 나타내는 특징 벡터(512차원)가 나오고, 같은 사람이면 벡터가 비슷합니다.

- **등록:** 사진을 넣으면 얼굴 벡터를 저장합니다. 영상을 넣으면 여러 프레임에서 벡터를 뽑아 함께 저장하므로 정면·옆얼굴·다른 조명에도 강해집니다.
- **등록 영상에 여러 명이 나올 때:** 가장 크게, 가장 오래 나온 얼굴을 자동으로 고르고, 화면에 후보 얼굴을 보여줘 사용자가 확인·변경할 수 있게 합니다.
- **비교:** 탐지된 얼굴의 벡터와 등록된 벡터의 유사도가 임계값보다 높으면 모자이크를 생략합니다. 영상에서는 추적 ID마다 한 번만 비교해 속도를 지킵니다.

5. 입출력 및 동작 흐름

구분	내용
입력	사진(jpg, png), 영상(mp4, 웹 데모 기준 1분·720p 이하), 실시간 영상 주소(RTSP·HTTP 스트림, 유튜브 라이브), (2단계) 제외할 인물의 사진 또는 영상
출력	얼굴이 모자이크된 사진/영상 (영상은 원본 소리 포함), 실시간 입력은 모자이크된 화면을 바로 표시 + 얼굴 수

```
사진 / 영상 파일 / 실시간 영상 주소 입력
  → YOLO11 얼굴 탐지
  → 추적 (영상·실시간, ByteTrack)
  → [2단계] InsightFace 제외 인물 필터
  → OpenCV 모자이크
  → 파일: 저장 (영상은 ffmpeg로 소리 병합)
    실시간: 화면에 바로 표시
  → 얼굴 수 표시
```

모듈 인터페이스 (0일 차 합의)

세 사람이 서로를 기다리지 않도록 함수의 입출력을 먼저 정합니다. 첫날은 사전학습 가중치로 detect_faces를 만들어 공유하고, 직접 학습한 모델은 8일 차에 best.pt만 교체합니다. 모델 로딩과 추적은 detect_faces 한 곳에서만 하고, 영상·실시간 처리는 이 함수를 프레임마다 호출합니다. 새 영상을 시작할 때는 reset_tracker()로 추적 상태를 초기화해 이전 영상의 추적 ID가 이어지지 않게 합니다. 데이터 타입, 예외 처리, 필요한 데이터 등 세부 규칙은 **역할별_구현명세.md**를 따릅니다.

```
# 공통: img = BGR uint8 배열, 박스 좌표는 픽셀 정수, 얼굴이 없으면 빈 리스트
# 박스 = {"xyxy": (x1,y1,x2,y2), "conf": 점수, "track_id": 번호 또는 None}

# A 이유노 (track=True: persist=True, tracker="bytetrack.yaml")
detect_faces(img, track=False) -> list[박스]
reset_tracker() -> None                    # 새 영상 시작 시 호출

# B 조영준
apply_mosaic(img, boxes) -> 새 img          # 입력 이미지는 그대로 둠
BoxHolder.update(boxes, frame_idx) -> list[박스]   # 끊긴 박스 유지
process_video(in_path, out_path, ref_embs=None) -> {faces, frames, fps, seconds}
process_stream(url, ref_embs=None) -> (프레임, 정보) 생성기

# C 나예린
list_candidates(photo_or_video) -> 후보 인물 목록 (점수순)
register_person(photo_or_video, index=0) -> ref_embs   # (N, 512)
filter_excluded(img, boxes, ref_embs, cache=None) -> 모자이크할 박스
```

6. 데이터 계획

- **필요 데이터:** 얼굴 위치에 바운딩 박스 라벨이 달린 이미지 (클래스 face 1개). 정면·옆얼굴, 큰 얼굴·작은 얼굴, 다인원, 마스크·모자·안경, 밝은 곳·어두운 곳 포함
- **출처:** AI Hub 안면 관련 데이터 (바운딩 박스 라벨 포함 여부 확인 필요, **1일 차에 바로 신청**), WIDER FACE (보조)
- **규모·분할:** 최소 1,000장 이상, 학습(train) 80% / 검증(val) 20%
- **라벨 변환:** AI Hub JSON을 YOLO txt(클래스, 중심x, 중심y, 너비, 높이를 0~1 비율로)로 변환. AI Hub와 WIDER FACE의 박스 기준(이마·턱 포함 범위)이 같은지 먼저 확인
- **별도 테스트셋:** 검증 세트는 학습 데이터와 분포가 같아 점수가 높게 나오므로, 실제 사용 환경(거리·브이로그 영상)의 프레임 약 200장으로 테스트셋을 따로 만들어 누락률을 측정
- **베이스라인 비교:** (A) 사전학습 YOLO 얼굴 모델 그대로 사용 vs (B) 위 데이터로 YOLO11을 직접 학습한 모델
- **인물 제외용 데이터:** 학습 데이터는 필요 없음. 테스트용으로 인터넷에 공개된 유명인 사진·영상만 준비 (출처 기록, 일반인 사진 제외)

7. 사용 도구

구분	도구
언어	Python
탐지 모델	YOLO11 (ultralytics, 착수 시점의 최신 버전 확인). 크기(n·s·m)는 정확도와 속도를 비교해 선택. 공개 YOLO11 얼굴 가중치(yolov11n-face, WIDER FACE 학습)로 시작한 뒤 직접 학습한 YOLO11로 교체
추적	ultralytics 내장 ByteTrack. 기본 추적기는 버전마다 다르므로 tracker="bytetrack.yaml"과 persist=True를 명시
인식 모델 (2단계)	InsightFace (사전학습 모델 그대로 사용)
이미지·영상 처리	OpenCV (얼굴 크기에 비례하는 블록 모자이크, 타원 마스크, 스트림 입력), ffmpeg (소리 병합, H.264 변환. Colab·리눅스 서버는 기본 제공, Windows는 별도 설치), yt-dlp (유튜브 라이브 주소 변환)
학습 환경	Google Colab 또는 학원 GPU 서버. 체크포인트는 Drive나 서버 저장소에 보관
웹 데모	Streamlit, Gradio, Vercel 중 C 나예린이 선택 (아래 비교 참고)

웹 방식	장점	주의할 점
Streamlit	화면과 모델을 파이썬 하나로 구성, 화면 구성이 자유로움	실시간 영상 표시는 직접 구현 필요
Gradio	업로드·웹캠·스트리밍 컴포넌트가 기본 제공, 가장 빠르게 데모 가능	화면 디자인 자유도가 낮음
Vercel	완성도 높은 웹 화면, 배포가 쉬움	모델을 실행할 수 없어 **모델 서버를 따로 둬야 함** (학원 GPU 서버, Hugging Face Spaces 등)

- **InsightFace 선택 이유:** face_recognition은 dlib 빌드가 필요해 설치가 까다로워 제외
- **탐지(detect) 방식 선택 이유:** 분할(segment)은 결과가 자연스럽지만 마스크 경계가 안쪽으로 잡히면 윤곽·귀가 노출되고, 얼굴 폴리곤 라벨이 없어 일정이 늘어남. 박스 탐지 + 타원 마스크 + 여백으로 자연스러움과 안전성을 함께 확보

8. 성공 기준 (목표 지표)

지표	목표값(제안)	측정 방법
탐지 정확도 mAP50	0.85 이상	검증 세트(val) 평가
누락률	10% 이하	별도 테스트셋에서 모자이크가 덮지 못한 얼굴 수 / 전체 얼굴 수 (영상은 프레임 단위). 프라이버시 목적상 가장 중요한 지표
영상 처리 속도	기기 성능에 따라 다름	사용 기기(Colab GPU, 학원 GPU 서버, CPU 노트북)별로 측정해 표로 기록. 모델 크기, imgsz, 720p, 읽기부터 저장까지 전체 기준을 함께 명시
실시간 처리	지연이 쌓이지 않음	실시간 주소 입력 시 처리 속도가 입력보다 느리면 밀린 프레임을 건너뛰어, 화면이 원본보다 계속 늦어지지 않는지 확인
인물 제외 오인식 (2단계)	0건	테스트 영상에서 제외 대상이 아닌 사람이 모자이크 없이 노출된 횟수
데모 시연	사진·영상·실시간 각 1건 이상 성공	발표 시 웹 화면에서 직접 시연

※ 목표값은 일반적인 수준으로 잡은 제안값이며, 실험 결과에 따라 조정합니다.

9. 역할 분담

담당	이름	역할	발표 파트
A	이유노	데이터 확보·라벨 변환, 테스트셋 구축, 평가 스크립트(mAP·누락률), YOLO11 직접 학습, 모델 크기 선택, 기기별 FPS 측정, 베이스라인 비교	성능
B	조영준	모자이크 처리, 영상 추적·박스 유지, 소리 병합·H.264 출력, 실시간 스트림 처리	시연
C	나예린	웹 방식 선택과 웹 데모, 특정 인물 제외(InsightFace), 사진·영상 인물 등록 UI, 발표 자료 구성	서비스

10. 진행 일정 (2주)

0일 차 (반나절, 전원): 모듈 인터페이스 합의, 사전학습 가중치로 detect_faces 공유, 테스트용 사진·영상과 실시간 테스트 주소를 공유 폴더에 정리

기간	A 이유노	B 조영준	C 나예린
1~2일	AI Hub 신청, WIDER FACE 다운로드, 라벨 변환 스크립트, 학원 GPU 서버 환경 확인	모자이크 함수: 비례 블록, 여백, 타원 마스크, 크기 0 박스 방어	웹 방식 결정, 뼈대: 사진 업로드 → 탐지 → 모자이크
3~4일	테스트셋 구축, 평가 스크립트 작성, 사전학습 모델 베이스라인 측정	영상 처리: detect_faces(track=True)로 프레임별 추적, 탐지가 끊긴 프레임의 박스 유지	InsightFace 등록(사진·영상), 얼굴 비교, 유사도 임계값 실험
5일	**중간 통합 (전원): 사진 전체 흐름 동작 확인**		
5~7일	YOLO11 직접 학습 (n·s·m 크기 비교, 체크포인트 저장)	ffmpeg 소리 병합, H.264 출력, 실시간 스트림 처리(프레임 건너뛰기·재연결)	track ID당 1회 인식으로 최적화, 등록 영상의 대상 인물 자동 선택, 오인식 측정
7일	**컷 판단 (전원): 인물 제외 오인식이 남으면 기능을 접고 C는 웹·영상 지원**		
8일	사전학습 vs 직접 학습 비교, 더 나은 모델을 best.pt로 교체, 기기별 FPS 측정표 작성	영상·실시간 탭 웹 연결, 버그 수정	인물 등록 UI, 영상·실시간에도 제외 기능 적용
8일	**최종 통합 + 기능 동결 (전원)**		
9~10일	성능 비교 표·그래프	시연용 전후 영상 제작, 실시간 시연 준비	발표 자료 구성, 데모 리허설

11. 예상 리스크와 대응

리스크	대응 방안
작은 얼굴·옆얼굴·가려진 얼굴 누락	다양한 각도·크기 데이터 추가, 필요 시 imgsz 확대, 박스에 여백을 두어 넉넉히 가림
작은 얼굴은 8×8 고정 모자이크로 가려지지 않음	블록 크기를 얼굴 크기에 비례하게 설정
영상에서 프레임마다 탐지가 끊겨 얼굴이 순간 노출	ByteTrack 추적으로 같은 사람을 이어 주고(persist=True 필수, 빠지면 프레임마다 ID가 새로 매겨짐), 탐지가 끊긴 몇 프레임 동안 마지막 박스 유지
영상 저장 시 소리가 사라짐 (OpenCV는 영상만 저장)	ffmpeg로 원본 소리를 다시 합침
ffmpeg가 없는 환경(Windows 로컬 PC)에서 소리 병합 실패	Colab·학원 서버에서 처리하거나, ffmpeg를 설치하고 PATH에 등록
사전학습 얼굴 가중치가 실사용 영상에서 약함	0일 차는 공개 yolov11n-face(WIDER FACE 학습)로 시작하고, 직접 학습한 YOLO11과 누락률을 비교해 교체
브라우저에서 결과 영상이 재생되지 않음	H.264 코덱으로 변환해 저장
실시간 처리가 입력보다 느려 지연이 쌓임	밀린 프레임은 건너뛰고 최신 프레임만 처리, 필요 시 작은 모델 크기 사용
실시간 스트림 끊김·유튜브 주소 직접 열기 불가	끊기면 자동 재연결, 유튜브 라이브는 yt-dlp로 실제 스트림 주소 변환
큰 모델 크기 사용 시 속도 저하	정확도와 FPS를 함께 비교해 크기 선택, 실시간용과 파일용 모델을 다르게 쓸 수 있게 설계
Vercel 선택 시 모델 실행 불가	화면은 Vercel, 모델은 학원 GPU 서버나 Hugging Face Spaces에 별도 배포
AI Hub 승인 지연·라벨 형식 차이	1일 차에 신청, 사전학습 모델을 우선 사용하고 WIDER FACE로 보완
두 데이터셋의 박스 기준 차이	변환 전에 샘플을 시각화해 기준 확인, 필요 시 한쪽 데이터셋만 사용
학습 중단 (Colab 세션 끊김, 서버 사용 시간 제한)	체크포인트를 주기적으로 저장하고 이어서 학습
로컬 PC 환경 불일치 (예: 이유노 PC는 numpy 1.18.5, ultralytics는 1.23 이상 필요)	개발·시연은 Colab, 학원 서버 또는 새 가상환경에서 진행
등록 영상에 여러 사람이 나옴	가장 크게·오래 나온 얼굴을 자동 선택하고, 사용자가 화면에서 확인·변경
인물 제외 오인식 (제외 대상이 아닌 사람이 노출)	유사도 임계값을 엄격하게 설정, 확신이 낮으면 모자이크 적용 (프라이버시 우선)
일정 초과	7일 차 컷 판단으로 2단계 기능을 접고 1단계 완성에 집중